DEDICATED TO INNOVATION

www.bacancy.com

BACANCY®

Customer Satisfaction Is Our Highest Priority

Employee Matters

In this session

What we are learning in this session

- Advanced Inheritance and Polymorphism
- SOLID principles

Let’s Start

Advanced Inheritance & Polymorphism

- **New Keyword :** The new keyword is used to indicate that a method or property in a derived class is intended to hide, rather than override, a method or property in the base class.
- It's important to note that the new keyword can be used to hide any method or property, regardless of whether it's virtual or not.
- The base class method can still be accessed using a base class reference.

Usage:

- When you don’t want to override but just define a new implementation of a method in the derived class.
- When both versions of the method should be accessible based on the reference type.

Advanced Inheritance & Polymorphism

```
class Parent
{
    public void Show()
    {
        Console.WriteLine("Parent class Show method");
    }
    public virtual void display()
    {
        Console.WriteLine("Parent class display method");
    }

}

class Child : Parent
{
    public new void Show()  // Hides the Parent's Show() method
    {
        Console.WriteLine("Child class Show method");
    }
    public override void display()  // Overrides the Parent's display() method
    {
        Console.WriteLine("Child class display method");
    }
}
```

```
static void Main(string[] args)
{
    Parent parent = new Parent();
    parent.Show();
    parent.display();

    Child child = new Child();
    child.Show();
    child.display();

    Parent parentRef = new Child();
    parentRef.Show();
    parentRef.display();
}
```

Difference between Method Overriding and Method Hiding

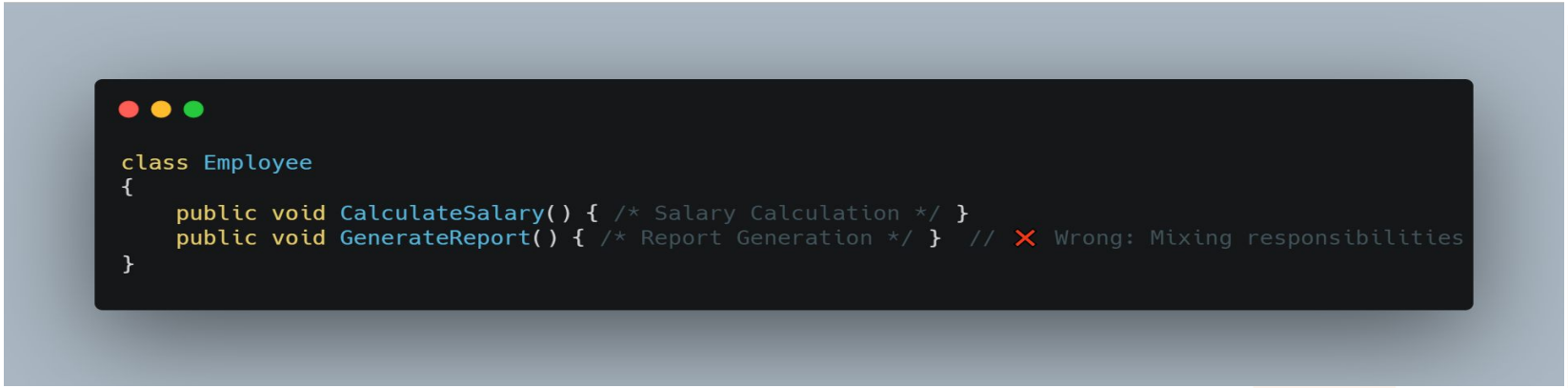
Method Overriding	Method Hiding (new keyword)
Runtime binding (based on object type)	**Compile-time binding** (based on reference type)
Derived class method is called even with base class reference (polymorphism)	**Base class method is called** when using base class reference
If many implementations are of the same kind and use common behavior, then it is superior to use abstract class.	If many implementations only share methods, then it is superior to use Interface.
Used when a derived class wants to **modify base class behavior dynamically**	Used when a derived class wants to define a **completely new independent method**
When the parent class reference variable points to the child class, then the method in the child class is invoked while calling.	When the parent class reference variable points to the child class, then the hidden method in the parent class is invoked while calling.

SOLID Principles

- SOLID is a set of five principles in object-oriented programming (OOP) that help developers write clean, scalable, and maintainable code.
- SOLID Design Principles represent five Design Principles used to make software designs more understandable, flexible, and maintainable. The Five SOLID Design Principles are as follows:
  1. SRP – Single Responsibility Principle
  2. OSP – Open – Closed Principle
  3. LSP – Liskov Substitution Principle
  4. ISP – Interface Segregation Principle
  5. DIP – Dependency Inversion Principle
- Applying the SOLID principles in our Application Development can make our application code more modular, easier to understand, and less error-prone when changes are made.

Single Responsibility Principle

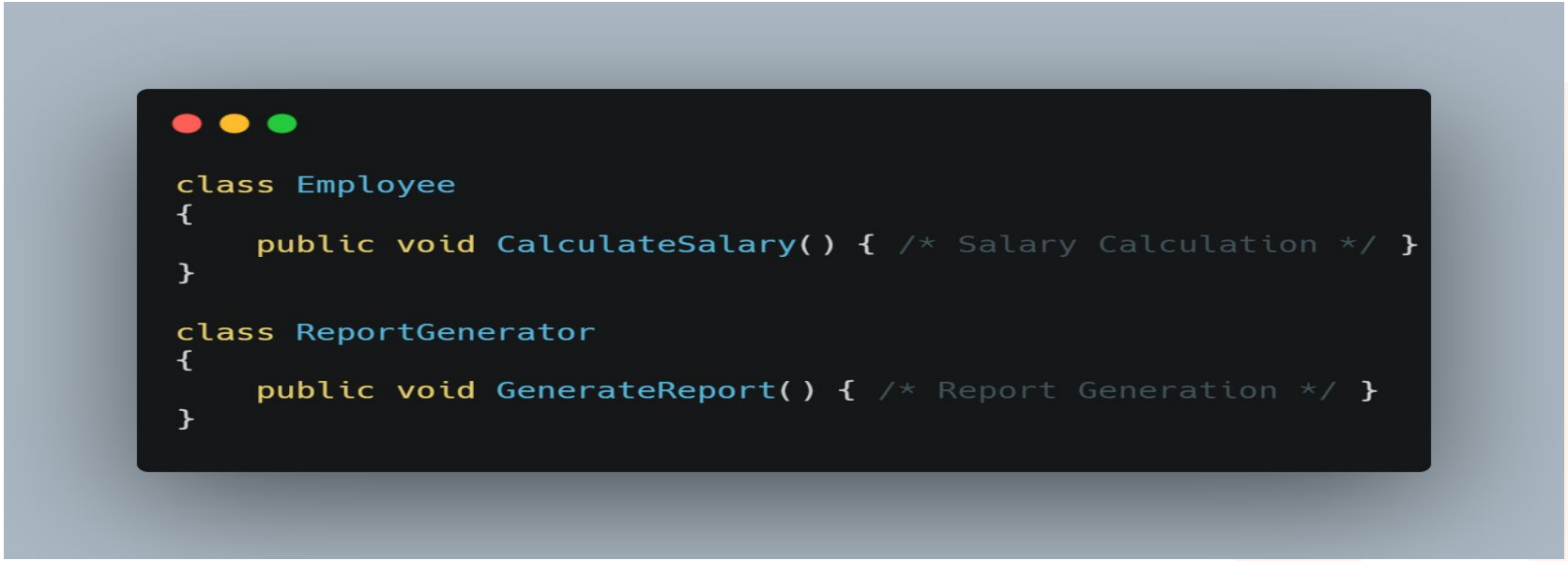
- The SRP states that a class should have only one reason to change, meaning it should have only one responsibility. This promotes modularization and makes the code easier to understand and maintain.



Problem: The class is responsible for both salary calculation and report generation, which are unrelated.

Single Responsibility Principle

- Employee handle salaries and ReportGenerator handles reports (Sepeartion of concerns).



```
class Employee
{
    public void CalculateSalary() { /* Salary Calculation */ }
}

class ReportGenerator
{
    public void GenerateReport() { /* Report Generation */ }
}
```

Open-Closed Principle

- Software entities (classes, modules, functions) should be open for extension but closed for modification.
- Once a class is written, it should be closed for modifications but open for extensions.

```
class Invoice
{
    public void ProcessInvoice(string type)
    {
        if (type == "Online") { /* Process Online Invoice */ }
        else if (type == "Offline") { /* Process Offline Invoice */ }  // ❌ Wrong: Modifying code for
new types
    }
}
```

Problem: Every time a new invoice type is added, we need to modify the class, which breaks OCP.

Open-Closed Principle

- New invoice types can be added **without modifying existing code**, only by extending the base class.

```
abstract class Invoice
{
    public abstract void ProcessInvoice();
}

class OnlineInvoice : Invoice
{
    public override void ProcessInvoice() { /* Process Online Invoice */ }
}

class OfflineInvoice : Invoice
{
    public override void ProcessInvoice() { /* Process Offline Invoice */ }
}
```

Liskov Substitution Principle

- Derived classes should be replaceable for their base classes without affecting correctness.
- A subclass should be able to replace its superclass **without causing errors**.

```
public class Bird
{
    public virtual void Fly()
    {
        Console.WriteLine("This bird is flying.");
    }
}

public class Sparrow : Bird { } // ✅ Sparrow can fly, no problem

public class Penguin : Bird
{
    public override void Fly() // ❌ Penguins can't fly, but forced to implement
    {
        throw new NotImplementedException("Penguins cannot fly!");
    }
}
```

```
class Program
{
    static void Main()
    {
        Bird bird1 = new Sparrow();
        bird1.Fly(); // ✅ Works fine

        Bird bird2 = new Penguin();
        bird2.Fly(); // ❌ Throws exception at runtime!
    }
}
```

Liskov Substitution Principle

Problem :

- Penguin inherits from Bird but cannot fly, so Fly() throws an exception.
- This violates LSP because a Penguin cannot fully substitute a Bird without breaking behavior.

Solution :

- Instead of forcing all birds to have a Fly() method, we should create separate interfaces for Flying Birds and Non-Flying Birds.

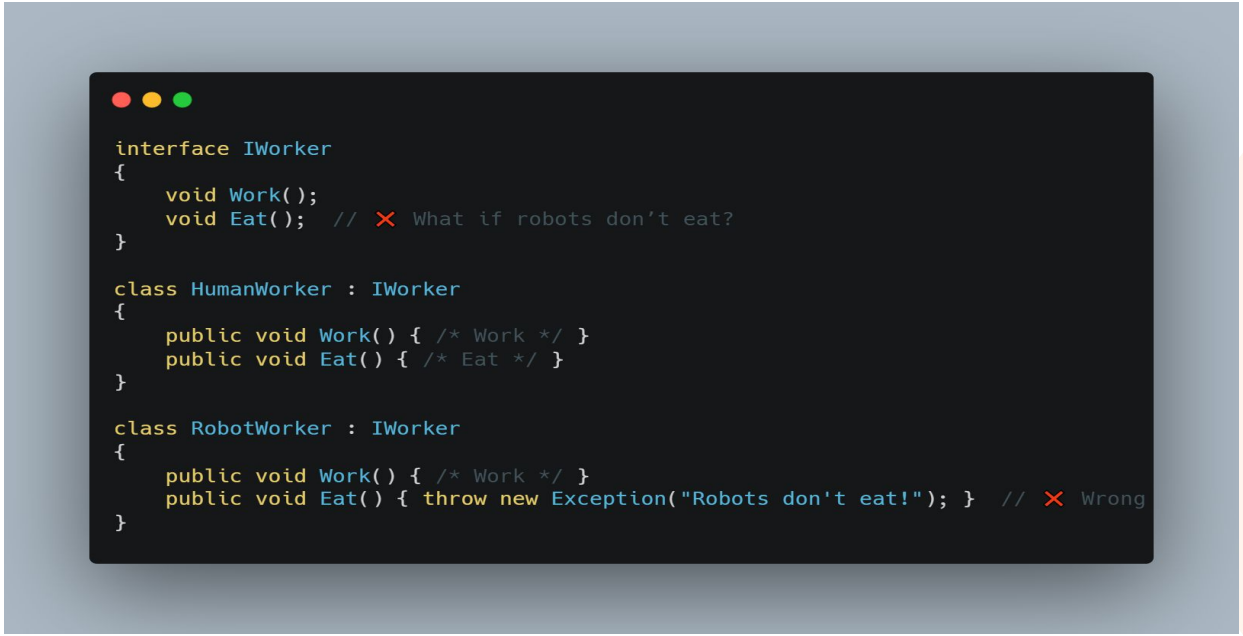
Liskov Substitution Principle

```
public class Bird
{
    public void Eat()
    {
        Console.WriteLine("This bird is eating.");
    }
}

public interface IFlyable
{
    void Fly();
}

public class Sparrow : Bird, IFlyable
{
    public void Fly()
    {
        Console.WriteLine("Sparrow is flying.");
    }
}

public class Penguin : Bird
{
    public void Swim()
    {
        Console.WriteLine("Penguin is swimming.");
    }
}
```

```
class Program
{
    static void Main()
    {
        IFlyable flyingBird = new Sparrow();
        flyingBird.Fly(); // ✅ Works correctly

        Bird penguin = new Penguin();
        // penguin.Fly(); ❌ Not possible now, avoiding runtime errors!
    }
}
```

Interface Segregation Principle

- The Interface Segregation Principle states that a class should not be forced to implement interfaces it does not use.
- This principle encourages the creation of small, client-specific interfaces.



```
interface IWorker
{
    void Work();
    void Eat();  // ✕ What if robots don't eat?
}

class HumanWorker : IWorker
{
    public void Work() { /* Work */ }
    public void Eat() { /* Eat */ }
}

class RobotWorker : IWorker
{
    public void Work() { /* Work */ }
    public void Eat() { throw new Exception("Robots don't eat!"); }  // ✕ Wrong
}
```

Interface Segregation Principle

Problem :

- RobotWorker is forced to implement Eat(), which does not make sense for robots.

Solution:

- HumanWorker implements both IWorkable and IEatable.
- RobotWorker only implements IWorkable, avoiding the unnecessary Eat() method.

Interface Segregation Principle

```
// Separate interface for working behavior
public interface IWorkable
{
    void Work();
}

// Separate interface for eating behavior
public interface IEatable
{
    void Eat();
}

// HumanWorker implements both interfaces because humans can work and eat
public class HumanWorker : IWorkable, IEatable
{
    public void Work()
    {
        Console.WriteLine("Human is working.");
    }

    public void Eat()
    {
        Console.WriteLine("Human is eating.");
    }
}

// RobotWorker only implements IWorkable, avoiding unnecessary Eat() method.
public class RobotWorker : IWorkable
{
    public void Work()
    {
        Console.WriteLine("Robot is working.");
    }
}
```

Dependency Inversion Principle

- Dependency Inversion Principle (DIP) states that High-level modules should not depend on low-level modules. Both should depend on abstractions.
- Abstractions should not depend on details. Details should depend on abstractions.

```
public class EmailService
{
    public void SendEmail(string message)
    {
        Console.WriteLine("Sending Email: " + message);
    }
}

public class NotificationService
{
    private EmailService _emailService = new EmailService(); // ✕ Direct dependency

    public void NotifyUser()
    {
        _emailService.SendEmail("Order Shipped!");
    }
}
```

Dependency Inversion Principle

Problem:

- NotificationService depends on EmailService directly.
- If we want to switch to SMSService, we must modify NotificationService, violating Open/Closed Principle.

Solution:

- Use Abstraction (Interfaces or Abstract Classes)
- High-level modules should depend on this abstraction instead of concrete classes.
- Apply Dependency Injection (DI)
- Decouple High-Level and Low-Level Modules

Dependency Inversion Principle

```
public interface INotification
{
    void Send(string message);
}

public class EmailService : INotification
{
    public void Send(string message)
    {
        Console.WriteLine("Sending Email: " + message);
    }
}

public class SMSService : INotification
{
    public void Send(string message)
    {
        Console.WriteLine("Sending SMS: " + message);
    }
}

public class NotificationService
{
    private readonly INotification _notification;

    public NotificationService(INotification notification) // ✅ Inject Dependency
    {
        _notification = notification;
    }

    public void NotifyUser()
    {
        _notification.Send("Order Shipped!");
    }
}
```

```
class Program
{
    static void Main()
    {
        INotification notification = new EmailService(); // Easily switch to SMSService
        NotificationService notificationService = new NotificationService(notification);
        notificationService.NotifyUser();
    }
}
```

Thank you